

## Chapter 6. Code Execution on the JVM

The two main services that any JVM provides are memory management and an easy-to-use container for execution of application code. We covered garbage collection in some depth in Chapters [4](#) and [5](#), and in this chapter we turn to code execution.

---

### NOTE

Recall that the Java virtual machine specification, usually referred to as the VM spec, describes how a conforming Java implementation needs to behave.

---

The VM spec defines execution of Java bytecode in terms of an interpreter. However, broadly speaking, interpreted environments have unfavorable performance as compared to programming environments that execute machine code directly. Most production-grade modern Java environments solve this problem by providing dynamic compilation capability.

As we discussed in [Chapter 3](#), this ability is otherwise known as *just-in-time compilation*, or just *JIT compilation*. It is a mechanism by which the JVM keeps track of which methods are being executed to determine whether individual methods are eligible for compilation to directly executable code.

In this chapter, we start by laying out the basic lifecycle of a Java application, as it typically happens today. Then, we provide a brief overview of bytecode interpretation and why HotSpot is different from other interpreters that you may be familiar with.

We then turn to the basic concepts of JIT compilation and profile-guided optimization. We discuss the code cache and then introduce the basics of HotSpot's compilation subsystem.

Toward the end of the chapter, we discuss recent changes in the Java platform driven by a rethinking of how execution of Java programs should be handled. These developments have largely been prompted by the shift to cloud-deployed applications and the need to address the concerns of this new deployment environment.

# Lifecycle of a Traditional Java Application

Let's start by diving a little deeper into what actually happens when you type: `java HelloWorld` on a Unix-like system (such as Linux or Mac; similar remarks apply to Windows).

At a low level, standard process execution occurs to set up the JVM, which is a single process. The shell locates the JVM binary (e.g., possibly in `$JAVA_HOME/bin/java`) and starts a process corresponding to that binary, passing the arguments (including the entrypoint class name).

The newly started process analyzes the command-line flags and prepares for VM initialization, which will be customized via the flags (for heap size, GC, etc.). At this time the process probes the machine it is running on and examines various system parameters, such as how many CPU cores the machine has, how much memory, and what precise set of CPU instructions are available.

This very detailed information is used to customize and optimize how the JVM configures itself. For example, the JVM will use the number of cores to determine how many threads to use when garbage collection runs and to size the *common pool* of threads.

---

## TIP

The auto-probing and self-configuring behavior of the JVM is important to be aware of, as it affects how Java applications behave in containers. We will cover this further in [Chapter 8](#).

---

One key early step is to reserve an area of userspace memory (from the operating system, aka C, heap) equal to `mxm` (or the default value) for the Java heap—the area where all Java objects will be stored. Another vital step is to initialize a repository to store Java classes and associated meta-data in (known as *Metaspace* in HotSpot).

Then the VM itself is created, usually via the function `JNI_CreateJavaVM`, on a new thread for HotSpot. The VM's own threads—such as the GC threads and the threads that perform JIT compilation—also need to be started up.

As discussed earlier, the bootstrapping classes are prepared and then initialized. The first bytecodes are run and first objects are created as soon

as classes are loaded—e.g., in the class initializer (`static {}` blocks aka `clinit` methods) for the bootstrapping classes.

The significance of this is that the JVM’s basic functions—such as JIT compilation and GC—are running from very early in the lifecycle of the application. As the VM starts up, there may be some GC and JIT activity even before control reaches the entrypoint class. Once it does, then further classloading will happen as the application begins to execute and needs to run code from classes that are not present in the class metadata cache.

For most typical production applications, therefore, the startup phase is characterized by a spike in classloading, JIT, and GC activity while the application reaches a steady state. Once this has occurred, the amount of JIT and classloading usually drops sharply because:

- The entire “world” of classes that the application needs has been loaded.
- The set of methods that are called often have already been converted to machine code by the JIT compiler.

However, it is important to recognize that “steady state” does not mean “zero change.” It is perfectly normal for applications to experience further classloading and JIT activity—such as *deoptimization* and *reoptimization*. This can be caused when a rarely executed code path is encountered and causes a new class to be loaded.

One other important special case of the startup-steady-state model is sometimes referred to as “two-phase classloading.” This occurs in applications that use Spring and other similar dependency injection techniques.

In this case, the core framework classes are loaded first. After that, the framework examines the main application code and config to determine a graph of objects that need to be instantiated to activate the application. This triggers a second phase of classloading where the application code and its other dependencies are loaded.

The case of GC behavior is a little bit different. In an application that is not suffering any particular performance problem, the pattern of GC is also likely to change when the steady state is reached—but GC events will still occur.

This is because in Java applications (apart from a very few pathological use cases), objects are created, are alive for some time, and then are automatically collected—this is the entire point of automatic memory man-

agement. However, the pattern of steady state GC may well look very different to that of the startup phase.

The overall impression that you should be building up from this description is one of a highly dynamic runtime. Applications that are deployed on it display the runtime characteristics of a well-defined startup phase, followed by a steady state where small (but usually nonzero) amounts of change occur. The community has adopted the term *dynamic VM mode* for this traditional Java application lifecycle.

However, with the rise of Cloud Native Java, there has been increased interest in new operational and deployment modes for Java applications that are better suited to containers and the cloud, and that deviate from dynamic VM mode in various ways.

We will discuss these recent developments later in the chapter, but first we need to discuss the basics of how code is executed on the JVM.

## Overview of Bytecode Interpretation

As we saw briefly in [“Executing Bytecode”](#), the JVM interpreter operates as a stack machine. This means that, unlike with physical CPUs, there are no registers used as immediate holding areas for computation. Instead, all values to be operated on are placed on an *evaluation stack*, and the stack machine instructions work by transforming the value(s) at the top of the stack.

The JVM provides three primary areas to hold data:

- The *evaluation stack*, which is local to a particular method
- *Local variables* to temporarily store results (also local to methods)
- The *object heap*, which is shared between methods and between threads

A series of VM operations that use the evaluation stack to perform computation can be seen in Figures [6-1](#) through [6-5](#), as a form of pseudocode that should be instantly recognizable to Java programmers.

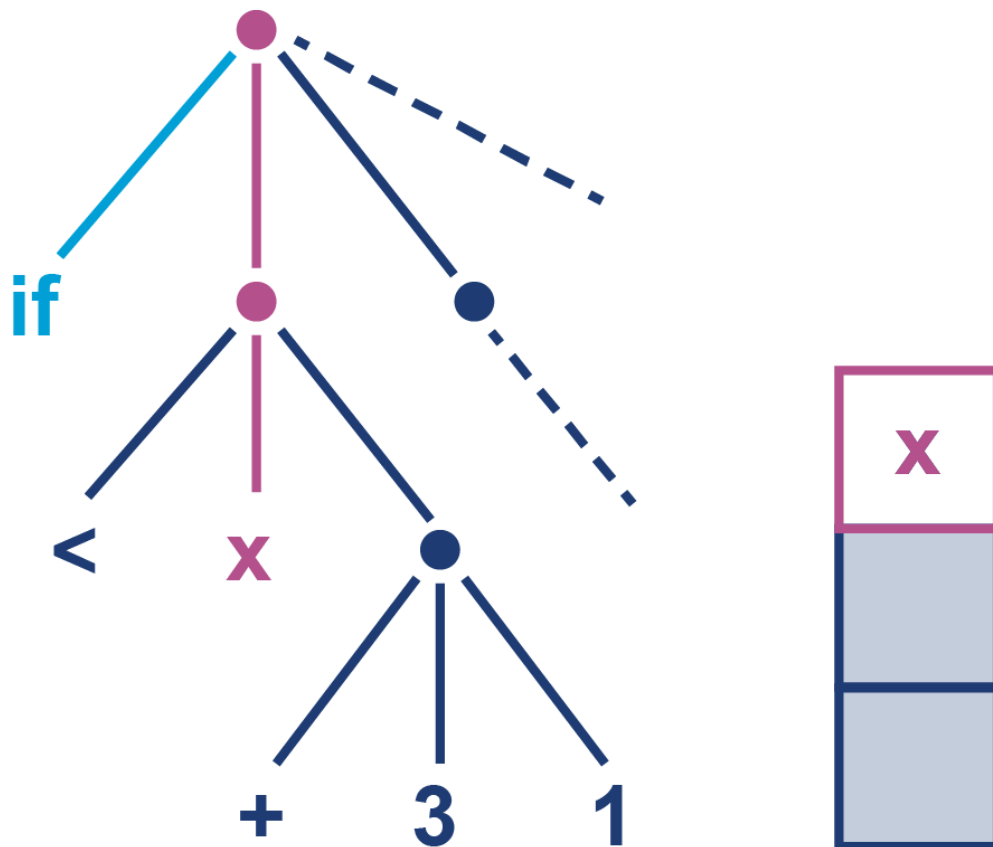


Figure 6-1. Initial interpretation state

The interpreter must now compute the righthand subtree to determine a value to compare with the contents of `x`.

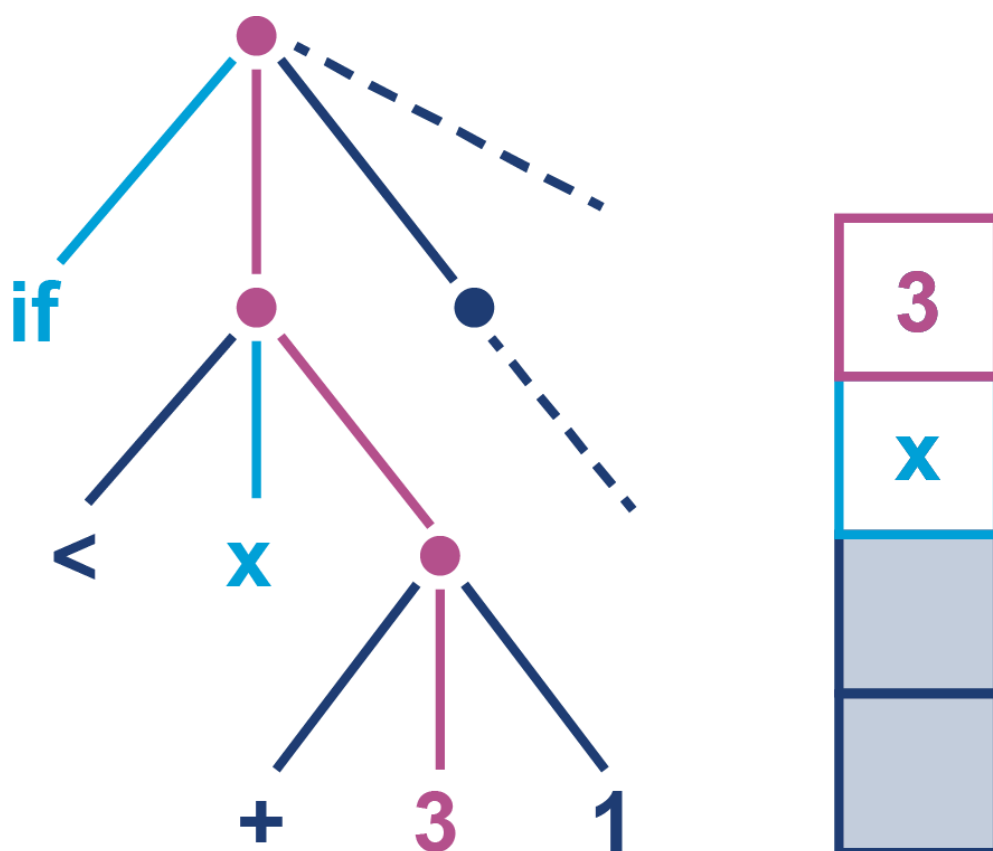


Figure 6-2. Subtree evaluation

The first value of the next subtree, an `int` constant `3`, is loaded onto the stack.

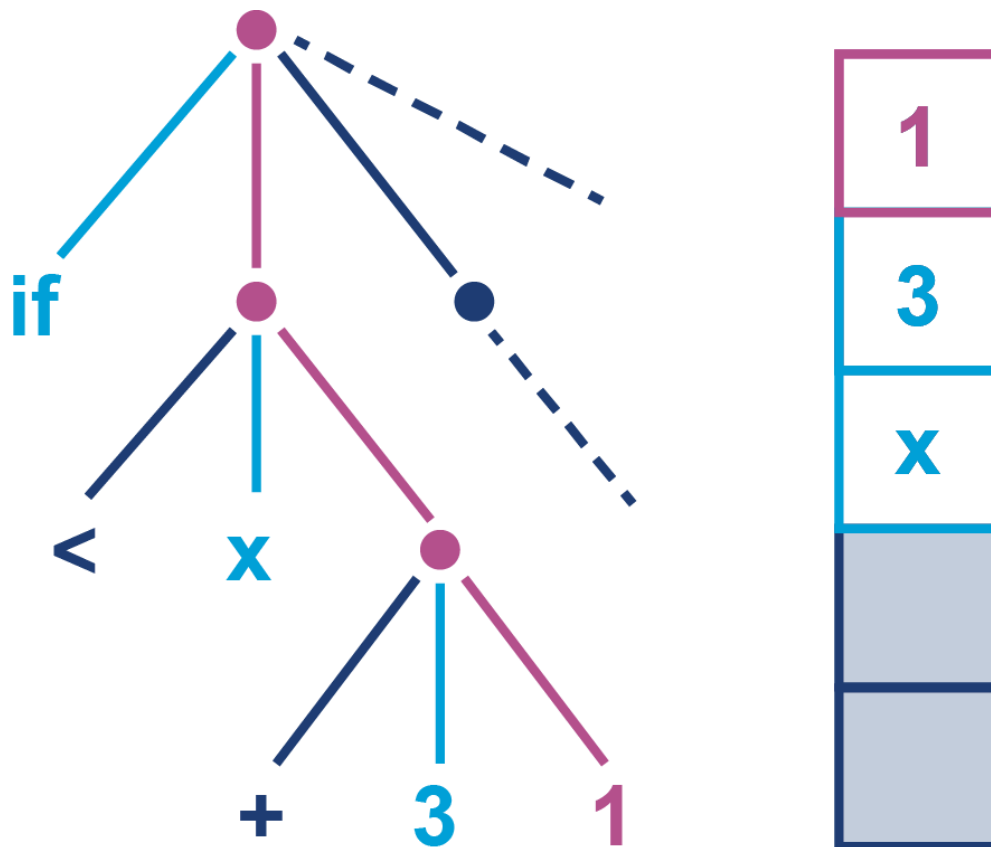


Figure 6-3. Subtree evaluation (Step 2)

Now another `int` value, `1`, is also loaded onto the stack. In a real JVM, these values will either have been loaded from the constants area of the class file or will use a “shortcut form” for common, small constants.

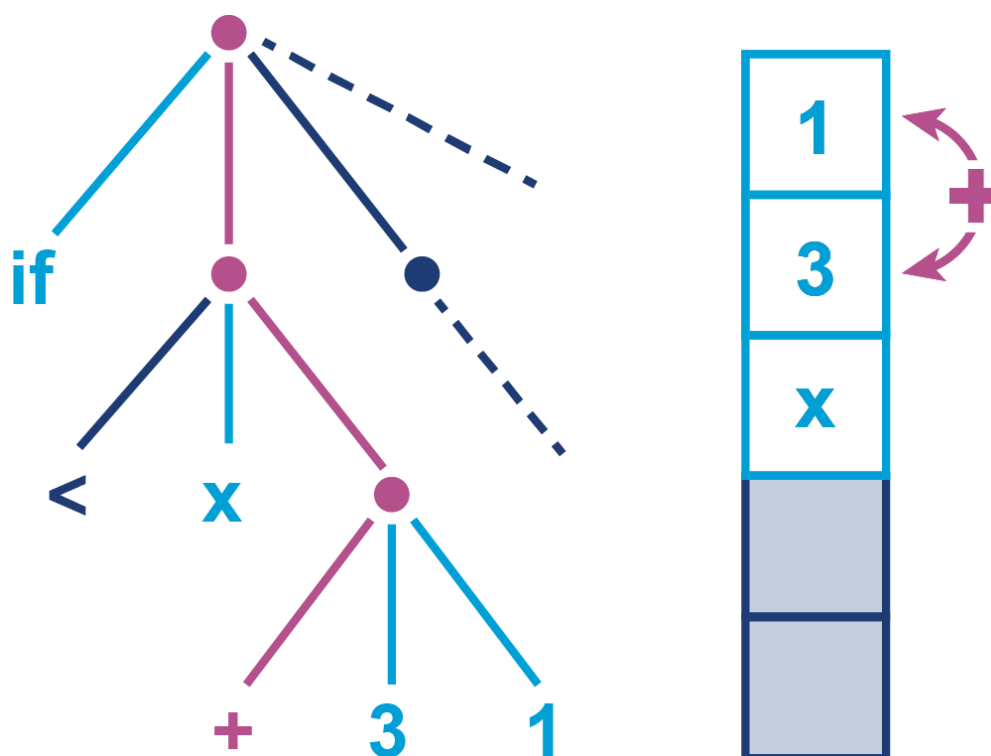


Figure 6-4. Subtree evaluation (Step 3)

At this point, the addition operation acts on the top two elements of the stack, removes them, and replaces them with the result of adding the two numbers together.

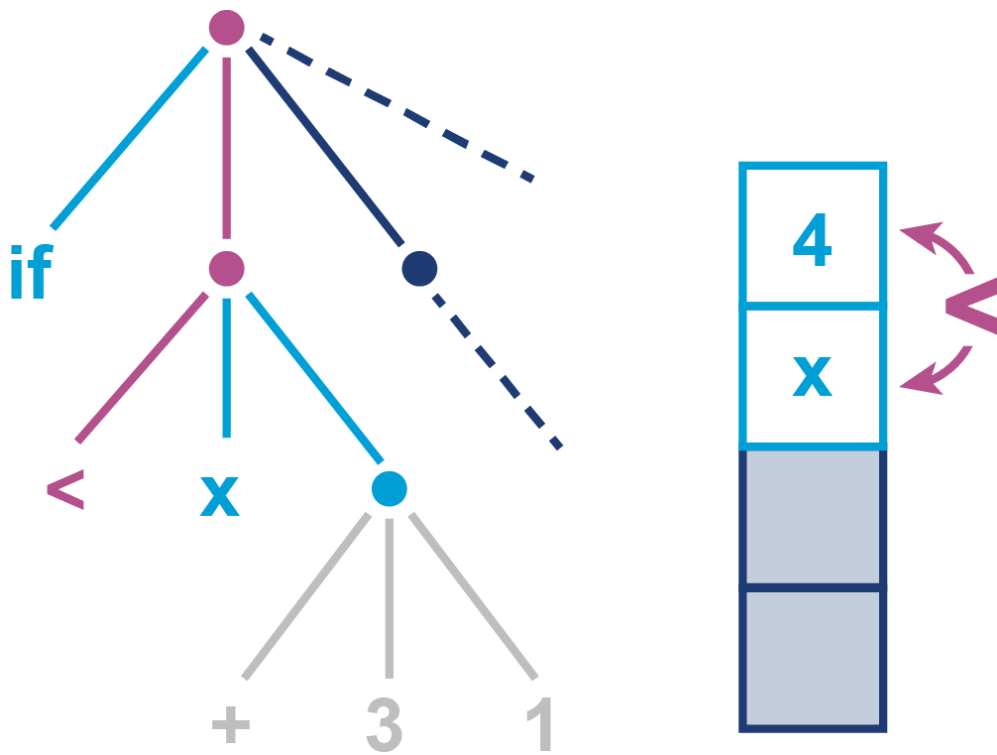


Figure 6-5. Final subtree evaluation

The resulting value is now available for comparison with the value contained in `x`, which has remained on the evaluation stack throughout the entire process of evaluating the other subtree.

## Introduction to JVM Bytecode

In the case of the JVM, each stack machine operation code (opcode) is represented by one byte, hence the name *bytecode*. Accordingly, opcodes run from 0 to 255, of which roughly 200 are in use as of Java 23.

Bytecode instructions are typed, in the sense that `iadd` and `dadd` expect to find the correct primitive types (two `int` and two `double` values, respectively) at the top two positions of the stack.

Many bytecode instructions come in “families,” with as many as one instruction for each primitive type and one for object references.

For example, within the `store` family, specific instructions have specific meanings: `dstore` means “store the top of stack into a local variable of type `double`,” whereas `astore` means “store the top of stack into a local variable of reference type.” In both cases, the type of the local variable must match the type of the incoming value.

As Java was designed to be highly portable, the JVM specification was designed to be able to run the same bytecode without modification on both *big-endian* and *little-endian* hardware architectures. As a result, JVM bytecode has to make a decision about which endianness convention to follow

(with the understanding that hardware with the opposite convention must handle the difference in software).

---

**TIP**

Bytecode is big-endian, so the most significant byte of any multi-byte sequence comes first.

---

Some opcode families, such as `load`, have *shortcut forms*. This allows the argument to be omitted, which saves the cost of the argument bytes in the class file. In particular, `aload_0` puts the current object (i.e., `this`) on the top of the stack. As that is such a common operation, this results in a considerable savings in class file size.

However, as Java classes are typically fairly compact, this design decision was probably more important in the early days of the platform, when class files—often applets—would be downloaded over a 14.4 Kbps modem.

---

**NOTE**

Since Java 1.0, only one new bytecode opcode (`invokedynamic`) has been introduced, and two (`jsr` and `ret`) have been deprecated.

---

The use of shortcut forms and type-specific instructions greatly inflates the number of opcodes that are needed, as several are used to represent the same conceptual operation. The number of assigned opcodes is thus much larger than the number of basic operations that bytecode represents, and bytecode is actually conceptually very simple.

Let's meet some of the main bytecode categories, arranged by opcode family. Note that in the tables that follow, `c1` indicates a two-byte constant pool index, whereas `i1` indicates a local variable in the current method. The parentheses indicate that the family has some opcodes that are shortcut forms.

The first category we'll meet is the *load and store* category, depicted in [Table 6-1](#). This category comprises the opcodes that move data on and off the stack—for example, by loading it from the constant pool or by storing the top of the stack into a field of an object in the heap.



Table 6-1. Load and store category

| Family name            | Arguments           | Description                                                                                     |
|------------------------|---------------------|-------------------------------------------------------------------------------------------------|
| <code>load</code>      | <code>( i1 )</code> | Loads value from local variable <code>i1</code> onto the stack                                  |
| <code>store</code>     | <code>( i1 )</code> | Stores top of stack into local variable <code>i1</code>                                         |
| <code>ldc</code>       | <code>c1</code>     | Loads value from <code>CP#c1</code> onto the stack                                              |
| <code>const</code>     |                     | Loads simple constant value onto the stack                                                      |
| <code>pop</code>       |                     | Discards value on top of stack                                                                  |
| <code>dup</code>       |                     | Duplicates value on top of stack                                                                |
| <code>getfield</code>  | <code>c1</code>     | Loads value from field indicated by <code>CP#c1</code> in object on top of stack onto the stack |
| <code>putfield</code>  | <code>c1</code>     | Stores value from top of stack into field indicated by <code>CP#c1</code>                       |
| <code>getstatic</code> | <code>c1</code>     | Loads value from static field indicated by <code>CP#c1</code> onto the stack                    |
| <code>putstatic</code> | <code>c1</code>     | Stores value from top of stack into static field indicated by <code>CP#c1</code>                |

The difference between `ldc` and `const` should be made clear. The `ldc` bytecode loads a constant from the constant pool of the current class. This holds strings, primitive constants, class literals, and other (internal) constants needed for the program to run.<sup>1</sup>

The `const` opcodes, on the other hand, take no parameters and are concerned with loading a finite number of true constants, such as `aconst_null`, `dconst_0`, and `iconst_m1` (the latter of which loads `-1` as an `int`).

The next category, the *arithmetic bytecodes*, apply only to primitive types, and none of them take arguments, as they represent purely stack-based operations. This simple category is shown in [Table 6-2](#).

Table 6-2. Arithmetic category

| Family name           | Description                                                      |
|-----------------------|------------------------------------------------------------------|
| <code>add</code>      | Adds two values from top of stack                                |
| <code>sub</code>      | Subtracts two values from top of stack                           |
| <code>div</code>      | Divides two values from top of stack                             |
| <code>mul</code>      | Multiplies two values from top of stack                          |
| <code>( cast )</code> | Casts value at top of stack to a different primitive type        |
| <code>neg</code>      | Negates value at top of stack                                    |
| <code>rem</code>      | Computes remainder (integer division) of top two values on stack |

In [Table 6-3](#), we can see the *flow control* category. This category represents the bytecode-level representation of the looping and branching constructs of source-level languages. For example, Java’s `for`, `if`, `while`, and `switch` statements will all be transformed into flow control opcodes after source code compilation.

Table 6-3. Flow control category

| Family name               | Arguments           | Description                                                                                    |
|---------------------------|---------------------|------------------------------------------------------------------------------------------------|
| <code>if</code>           | <code>( i1 )</code> | Branch to the location indicated by the argument if the condition is <code>true</code>         |
| <code>goto</code>         | <code>i1</code>     | Unconditional branch to the supplied offset                                                    |
| <code>return</code>       |                     | Returns from the current method to caller, passing back the value on top of the stack (if any) |
| <code>tableswitch</code>  |                     | Jump to a location specified in a jump table, using a label as the jump index                  |
| <code>lookupswitch</code> |                     | Jump to a location specified in a jump table, using a key for the jump index                   |

---

#### NOTE

A detailed description of how `tableswitch` and `lookupswitch` operate is outside the scope of this book. You can refer to the latest [Java virtual machine specification](#) for more details.

---

The flow control category seems very small, but the true count of flow control opcodes is surprisingly large. This is due to there being a large number of members of the `if` opcode family. We met the `if_icmpge` opcode (*if-integer-compare-greater-or-equal*) in the `javap` example back in [Chapter 3](#), but there are many others that represent different variations of the Java `if` statement.

The deprecated `jsr` and `ret` bytecodes (which have not been output by `javac` since Java 6) are also part of this family. They are no longer legal for modern versions of the platform so have not been included in this table.

One of the most important categories of opcodes is shown in [Table 6-4](#). This is the *method invocation* category, the only mechanism that the Java program allows to transfer control to a new method. That is, the platform completely separates the concepts of local flow control (i.e., within a method) and transfer of control to another method.

Table 6-4. Method invocation category

| Opcode name                  | Arguments                                             | Description                                                                             |
|------------------------------|-------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <code>invokevirtual</code>   | <code>c1</code>                                       | Invokes the method found at CP# <code>c1</code> via virtual dispatch                    |
| <code>invokespecial</code>   | <code>c1</code>                                       | Invokes the method found at CP# <code>c1</code> via “special” (i.e., exact) dispatch    |
| <code>invokeinterface</code> | <code>c1</code> , <code>count</code> , <code>0</code> | Invokes the interface method found at CP# <code>c1</code> using interface offset lookup |
| <code>invokestatic</code>    | <code>c1</code>                                       | Invokes the static method found at CP# <code>c1</code>                                  |
| <code>invokedynamic</code>   | <code>c1</code> , <code>0</code> , <code>0</code>     | Dynamically looks up which method to invoke and executes it                             |

The JVM’s design—and use of explicit *method call* opcodes—means that there is no equivalent of a `call` operation as found in machine code.

Instead, JVM bytecode uses some specialist terminology; we speak of a *call site*, which is a place within a method (the caller) where another method (the callee) is called. Not only that, but in the case of a nonstatic method call, there is always some object that we resolve the method upon. This object is known as the *receiver object*, and its runtime type is referred to as the *receiver type*.

---

**NOTE**

Calls to static methods are always turned into `invokestatic` and have no receiver object.

---

Java programmers who are new to looking at the VM level may be surprised to learn that method calls on Java objects are actually transformed into one of three possible bytecodes (`invokevirtual`, `invokespecial`, or `invokeinterface`), depending on the context of the call.

---

**TIP**

It can be a very useful exercise to write some Java code and see what circumstances produce each possibility, by disassembling a simple Java class with `javap`.

---

Instance method calls are normally turned into `invokevirtual` instructions, except when the static type of a receiver object is only known to be an interface type. In this case, the call is instead represented by an `invokeinterface` opcode. Finally, in the cases (e.g., private methods or superclass calls) where the exact method for dispatch is known at compile time, an `invokespecial` instruction is produced.

This raises the question of how `invokedynamic` enters the picture. The short answer is that there is no direct language-level access to `invokedynamic` in Java, even as of version 23.

In fact, when `invokedynamic` was added to the runtime in Java 7, there was no way at all to force `javac` to emit the new bytecode. In this old version of Java, the `invokedynamic` technology had only been added to support long-term experimentation and non-Java dynamic languages (especially JRuby).

However, from Java 8 onward, `invokedynamic` has become a crucial part of the Java language, and it is used to provide support for advanced language features. Let's take a look at a simple example from Java 8 lambdas:

```
public class LambdaExample {
    private static final String HELLO = "Hello";

    public static void main(String[] args) throws Exception {
        Runnable r = () -> System.out.println(HELLO);
        Thread t = new Thread(r);
        t.start();
    }
}
```

```

        t.join();
    }
}

```

This trivial usage of a lambda expression produces bytecode as shown:

```

public static void main(java.lang.String[]) throws java.lang.Exception;
Code:
    0: invokedynamic #2, 0 // InvokeDynamic #0:run:()Ljava/lang/Runnable;
    5: astore_1
    6: new          #3      // class java/lang/Thread
    9: dup
   10: aload_1
   11: invokespecial #4      // Method java/lang/Thread.<init>:(Ljava/lang/Runnable;)V
   14: astore_2
   15: aload_2
   16: invokevirtual #5      // Method java/lang/Thread.start:()V
   19: aload_2
   20: invokevirtual #6      // Method java/lang/Thread.join:()V
   23: return

```

Even if we know nothing else about it, the form of the `invokedynamic` instruction indicates that some method is being called, and the return value of that call is placed upon the stack.

Digging further into the bytecode we discover that, unsurprisingly, this value is the object reference corresponding to the lambda expression. It is created by a platform factory method that is being called by the `invokedynamic` instruction. This invocation makes reference to extended entries in the constant pool of the class to support the dynamic runtime nature of the call.

This is perhaps the most obvious use case of `invokedynamic` for Java programmers, but it is not the only one. This opcode is extensively used by non-Java languages on the JVM, such as Kotlin, JRuby, and Scala, and increasingly by Java frameworks as well. We will meet some related aspects of `invokedynamic` later in the book.

The final category of opcodes we'll consider are the *platform opcodes*. They are shown in [Table 6-5](#), and comprise operations such as allocating new heap storage and manipulating the intrinsic locks (the monitors used by synchronization) on individual objects.

Table 6-5. Platform opcodes category

| Opcode name                                | Arguments         | Description                                                              |
|--------------------------------------------|-------------------|--------------------------------------------------------------------------|
| <code>new</code>                           | <code>c1</code>   | Allocates space for an object of type found at CP# <code>c1</code>       |
| <code>newarray</code>                      | <code>prim</code> | Allocates space for a primitive array of type <code>prim</code>          |
| <code>anewarray</code>                     | <code>c1</code>   | Allocates space for an object array of type found at CP# <code>c1</code> |
| <code>arraylength</code>                   |                   | Replaces array on top of stack with its length                           |
| <code>monitorente</code><br><code>r</code> |                   | Locks monitor of object on top of stack                                  |
| <code>monitorexit</code><br><code>t</code> |                   | Unlocks monitor of object on top of stack                                |

For `newarray` and `anewarray` the length of the array being allocated needs to be on top of the stack when the opcode executes.

In the catalog of bytecodes, there is a clear difference between “coarse” and “fine-grained” bytecodes, in terms of the complexity required to implement each opcode.

For example, arithmetic operations will be very fine-grained and are implemented in pure assembly in HotSpot. By contrast, coarse operations (e.g., operations requiring constant pool lookups, especially method dispatch) will need to call back into the HotSpot VM.

Along with the semantics of individual bytecodes, we should also say a word about safepoints in interpreted code. In [Chapter 5](#) we met the concept of a JVM safepoint, as a point where the JVM needs to perform some housekeeping and requires a consistent internal state. This includes the object graph (which is, of course, being altered by the running application threads in a very general way).

To achieve this consistent state, all application threads must be stopped to prevent them from mutating the shared heap for the duration of the JVM’s housekeeping. How is this done?

The solution is to recall that every JVM application thread is a true OS thread.<sup>2</sup> Not only that, but for threads executing interpreted methods, when an opcode is about to be dispatched the application thread is definitely running JVM interpreter code, not user code. The heap should,

therefore, be in a consistent state, and the application thread can be stopped.

Therefore, “in between bytecodes” is an ideal time to stop an application thread, and it is one of the simplest examples of a safepoint.<sup>3</sup>

The situation for JIT-compiled methods is more complex, but essentially equivalent barriers must be inserted into the generated machine code by the JIT compiler.

## Simple Interpreters

As mentioned in [Chapter 3](#), the simplest interpreter can be thought of as a `switch` statement inside a `while` loop. Let’s look at a simple example of this type of interpreter, written in Java, that can execute a small subset of JVM bytecode.<sup>4</sup>

The `execMethod()` method of the interpreter interprets a single method of bytecode. Just enough opcodes have been implemented (some of them with dummy implementations) to allow integer math and “Hello World” to run.

A full implementation capable of handling even a very simple program would require complex operations, such as constant pool lookup, to have been implemented and work properly. However, even with only some very bare bones available, the basic structure of the interpreter is clear:

```
public EvalValue execMethod(final byte[] instr) {
    if (instr == null || instr.length == 0)
        return null;

    EvaluationStack eval = new EvaluationStack();

    int current = 0;
    LOOP:
    while (true) {
        byte b = instr[current++];
        Opcode op = table[b & 0xff];
        if (op == null) {
            System.err.println("Unrecognized opcode byte: " + (b & 0xff));
            System.exit(1);
        }
        byte num = op.numParams();
        switch (op) {
            case IADD:
                eval.iadd();
                break;
            case ICONST_0:
```

```

        eval.iconst(0);
        break;
    // ...

    case IRETURN:
        return eval.pop();
    case ISTORE:
        istore(instr[current++]);
        break;
    case ISUB:
        eval.isub();
        break;
    // Dummy implementation
    case ALOAD:
    case ALOAD_0:
    case ASTORE:
    case GETSTATIC:
    case INVOKEVIRTUAL:
    case LDC:
        System.out.print("Executing " + op + " with param bytes:");
        for (int i = current; i < current + num; i++) {
            System.out.print(instr[i] + " ");
        }
        current += num;
        System.out.println();
        break;
    case RETURN:
        return null;
    default:
        System.err.println("Saw " + op + " : can't happen. Exit.");
        System.exit(1);
    }
}
}

```

Bytecodes are read one at a time from the method and are dispatched based on the code. In the case of opcodes with parameters, these are read from the stream as well, to ensure that the read position remains correct.

Temporary values are evaluated on the `EvaluationStack`, which is a local variable in `execMethod()`. The arithmetic opcodes operate on this stack to perform the calculation of integer math.

Method invocation is not implemented in the simplest version of the interpreter—but if it were, then it would proceed by looking up a method in the constant pool, finding the bytecode corresponding to the method to be invoked, and then recursively calling `execMethod()`.



## HotSpot-Specific Details

HotSpot is a production-quality JVM with extensive advanced features designed to enable fast execution, even in interpreted mode. Rather than the simple style that we met in the simple interpreter example, HotSpot is a *template interpreter*, which generates the interpreter dynamically each time it is started up.

This is significantly more complex to understand and makes reading even the interpreter source code a challenge for the newcomer. HotSpot also uses a relatively large amount of assembly language to implement the simple VM operations (such as arithmetic) and exploits the native platform stack frame layout for further performance gains.

Also potentially surprising is that HotSpot defines and uses JVM-specific (aka private) bytecodes that do not appear in the VM spec. These are used to allow HotSpot to differentiate common hot cases from the more general use case of a particular opcode.

This is designed to help deal with a surprising number of edge cases. For example, a `final` method cannot be overridden, so the developer might think an `invokespecial` opcode would be emitted by `javac` when such a method is called.

However, the Java Language Specification has something to say about this case:

*Changing a method that is declared `final` to no longer be declared `final` does not break compatibility with pre-existing binaries.*

—JLS 13.4.17

Consider a piece of Java code such as:

```
public class A {
    public final void fMethod() {
        // ... do something
    }
}

public class CallA {
    public void otherMethod(A obj) {
        obj.fMethod();
    }
}
```

Now, suppose `javac` compiled calls to `final` methods into `invokespecial`. The bytecode for `CallA::otherMethod` would look something like this:

```
public void otherMethod()  
Code:  
  0: aload_1  
  1: invokespecial #4           // Method A.fMethod:()V  
  4: return
```

Now, suppose the code for `A` changes so that `fMethod()` is made nonfinal. It can now be overridden in a subclass; we'll call it `B`. Now suppose that an instance of `B` is passed to `otherMethod()`. From the bytecode, the `invokespecial` instruction will be executed, and the wrong implementation of the method will be called.

This is a violation of the rules of Java's object orientation. Strictly speaking, it violates the *Liskov substitution principle* (named for Barbara Liskov, one of the pioneers of object-oriented programming), which, simply stated, says that an instance of a subclass can be used anywhere that an instance of a superclass is expected. This principle is also the *L* in the famous SOLID principles of software engineering.

For this reason, calls to `final` methods must be compiled into `invokevirtual` instructions. However, because the JVM knows that such methods cannot be overridden, the HotSpot interpreter has a private bytecode that is used exclusively for dispatching `final` methods. The private bytecode approach in HotSpot is a performance optimization enabling a compile time statically bound function call. Without this, a dynamic call at runtime would be required to determine the correct function call.

To give another example, the language specification says that an object that is subject to finalization must register with the finalization subsystem. This registration must occur immediately after the supercall to the `Object` constructor `Object::<init>` has completed. In the case of JVMTI and other potential rewritings of the bytecode, this code location may become obscured. To ensure strict conformance, HotSpot has a private bytecode that marks the return from the original `Object` constructor.

A list of the opcodes can be found in `hotspot/src/share/vm/interpreter/bytecodes.cpp`, and the HotSpot-specific special cases are listed there as "JVM bytecodes."

With a basic understanding of how the JVM interprets bytecode, let's move on to look at how it utilizes JIT compilation.

## JIT Compilation in HotSpot

Just-in-time compilation is a general technique whereby programs are converted (usually from some convenient intermediate format) into highly optimized machine code at runtime. HotSpot and other mainstream production-grade JVMs rely heavily on this approach.

To be efficient, the technique uses runtime information to guide the optimization process. This is known as *profile-guided optimization* (PGO), and we'll turn to this subject next.

### Profile-Guided Optimization

In its simplest form, PGO gathers information about your program at runtime and builds a profile that can be used to determine which parts of your program are used frequently and would benefit most from optimization—which in HotSpot means JIT compilation.

---

#### NOTE

The JIT subsystem shares VM resources with your running program, so the cost of this profiling and any optimizations performed needs to be balanced against the expected performance gains over the process lifetime.

---

The cost of compiling bytecode to native code is paid at runtime and consumes resources (CPU cycles, memory) that could otherwise be dedicated to executing your program. Therefore, JIT compilation is performed sparingly, and the VM collects statistics about your program (looking for “hot spots”) to know where best to optimize.

Recall the overall architecture that was shown back in [Figure 3-3](#): the profiling subsystem is keeping track of which methods are running. If a method crosses a threshold that makes it eligible for compilation, then the emitter subsystem fires up a compilation thread to convert the bytecode into machine code.

---

#### NOTE

The design of modern versions of `javac` is intended to produce “dumb bytecode.” It performs only very limited optimizations, and instead provides a representation of the program that is easy for the JIT compiler to understand.

---

In [Chapter 3](#), we introduced the problem of JVM warmup—which we now understand to be a result of PGO. This period of unstable performance when the application starts up has frequently led Java developers to ask questions such as, “Can’t we save the compiled code to disk and use it the next time the application starts up?” or “Isn’t it very wasteful to rerun optimization and compilation decisions every time we run the application?”

The problem is that these questions contain some assumptions about the nature of running application code. Let’s look at an example from the financial industry to illustrate the problem.

US unemployment figures are released once a month. This *nonfarm payroll* (NFP) day produces traffic in trading systems that is highly unusual and not normally seen throughout the rest of the month.

If optimizations had been saved from another day and run on NFP day, they would not be as effective as freshly calculated optimizations. This would have the end result of making a trading system that uses precomputed optimizations actually less competitive than an application using PGO.

---

**TIP**

PGO is not a one-way street, and compilation decisions can be adjusted at runtime based on a drop in runtime performance caused by a change in profile. However, with projects like GraalVM and Leyden as seen in [Chapter 15](#), the answer in the Java ecosystem to “Should we ahead-of-time compile or use PGO?” is more complex to answer. We will revisit this discussion as we look at newer technologies later.

---

This behavior, where application performance varies significantly between different runs of the application, is very common and represents the kind of domain information that an environment like Java is supposed to protect the developer from.

For this reason, HotSpot does not attempt to save any profiling information for application code and instead discards it when the VM shuts down, so the profile must be built again from scratch each time.

In the next section, we’ll dig a little deeper into the mechanics of how HotSpot implements JIT compilation, building on the discussion of object layout that we introduced in [Chapter 4](#).

# Klass Words, Vtables, and Pointer Swizzling

Let's start by recalling that HotSpot is a multithreaded C++ application. This might seem a simplistic statement, but it is worth remembering that as a consequence, every executing Java program is actually always part of a multithreaded application from the operating system's point of view. Even single-threaded Java apps are always executing alongside VM threads.

One of the most important groups of threads within HotSpot are the threads that comprise the JIT compilation subsystem. This includes profiling threads that detect when a method is eligible for compilation and the compiler threads themselves that generate the actual machine code.

The method-scoped nature of HotSpot's JIT compilation uses the virtual function tables (vtables) as a key part of the implementation.

The overall picture is that when compilation is indicated by the emitter subsystem, the method is placed on a compiler thread, which compiles in the background. The overall process can be seen in [Figure 6-6](#).

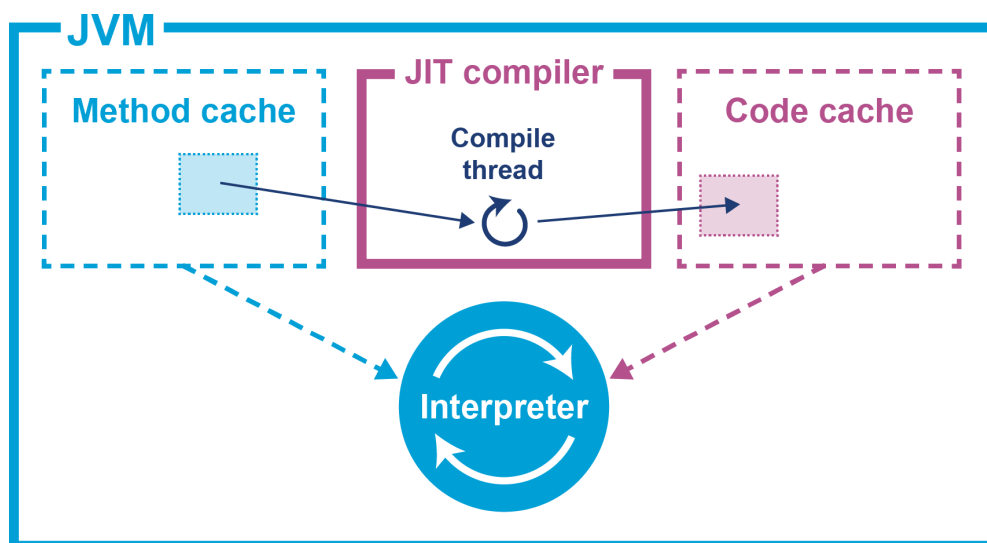


Figure 6-6. Simple compilation of a single method

When the optimized machine code is available, the entry in the vtable of the relevant klass is updated to point at the new compiled code.

---

## TIP

The vtable pointer updating is given the slightly strange name of *pointer swizzling*.

---

This means that any new calls to the method will get the compiled form, whereas threads that are currently executing the interpreted form will

finish the current invocation in interpreted mode but will pick up the new compiled form on the next call.

You should also know that the basic unit of compilation in HotSpot is a whole method, so all the bytecode corresponding to a single method is compiled into native code at once. However, HotSpot also supports compilation of a hot loop using a technique called *on-stack replacement* (OSR).

OSR is used to help the case where a method is not called frequently enough to be compiled but contains a loop that would be eligible for compilation if the loop body was a method in its own right.

When considering machine optimized code, it is important that the HotSpot optimizations are available on a given architecture to achieve the benefits. OpenJDK HotSpot has been widely ported to many different architectures, with x86, x86-64, and ARM being the primary targets. SPARC, Power, MIPS, and S390 are also supported to varying degrees. Oracle officially supports Linux, macOS, and Windows as operating systems, but there are open source projects to natively support a much wider selection, including BSDs and embedded systems.

## Compilers Within HotSpot

The HotSpot JVM actually has not one but two JIT compilers in it. These are properly known as C1 and C2, but they are sometimes referred to as the client compiler and the server compiler, respectively. Historically, C1 was used for GUI apps and other “client” programs, whereas C2 was used for long-running “server” applications. Modern Java apps generally blur this distinction, and HotSpot has changed to take advantage of the new landscape.

---

### NOTE

A compiled code unit is known as an *nmethod* (short for native method).

---

The general approach that both compilers take is to rely on a key measurement to trigger compilation: the number of times a method is invoked, or the *invocation count*. Once this counter hits a certain threshold, the VM is notified and will consider queuing the method for compilation.

The compilation process proceeds by first creating an internal representation of the method. Next, optimizations are applied that take into account profiling information that has been collected during the interpreted phase. However, the internal representation of the code that C1 and C2

produce is quite different. C1 is designed to be simpler and have shorter compile times than C2. The tradeoff is that, as a result, C1 does not optimize as fully as C2.

One technique that is common to both is *single static assignment*. This essentially converts the program to a form where no reassignment of variables occurs. In Java programming terms, the program is effectively rewritten to contain only `final` variables.

Historically, the JVM required the developer to choose between the C1 and C2 compilers at application startup time. However, since Java 6, the JVM has supported a mode called *tiered compilation*, which is the default for modern applications.

This is often loosely explained as running in interpreted mode until the simple C1 compiled form is available, then switching to using that compiled code while C2 completes more advanced optimizations.

However, this description is not completely accurate. From the *advanced-ThresholdPolicy.hpp* source file, we can see that within the VM there are five possible levels of execution:

- Level 0: interpreter
- Level 1: C1 with full optimization (no profiling)
- Level 2: C1 with invocation and backedge counters
- Level 3: C1 with full profiling
- Level 4: C2

We can also see in [Table 6-6](#) that not every level is utilized by each compilation approach.

Table 6-6. Compilation pathways

| Pathway | Description                                                             |
|---------|-------------------------------------------------------------------------|
| 0-3-4   | Interpreter, C1 with full profiling, C2                                 |
| 0-2-3-4 | Interpreter, C2 busy so quick-compile C1, then full-compile C1, then C2 |
| 0-3-1   | Trivial method                                                          |
| 0-4     | No tiered compilation (straight to C2)                                  |

In the case of the trivial method, the method starts off interpreted as usual but then C1 (with full profiling) is able to determine the method to be trivial. This means that it is clear that the C2 compiler would produce no better code than C1, so compilation terminates.

As noted, tiered compilation has been the default for some time now, and it is not normally necessary to adjust its operation during performance tuning. However, an awareness of its operation is still sometimes useful, as it can complicate the observed behavior of compiled methods and potentially mislead the unwary performance engineer.

## The Code Cache

JIT-compiled code is stored in a memory region called the *code cache*. This area also stores other native code belonging to the VM itself, such as parts of the interpreter.

The code cache has a fixed maximum size that is set at VM startup. It cannot expand past this limit, so it is possible for it to fill up. At this point, no further JIT compilations are possible, and any remaining uncompiled code will execute only in the interpreter. This will have an impact on performance and can result in the application being significantly less performant than the potential maximum.

Internally, the code cache is implemented as a heap containing an unallocated region and a linked list of freed blocks. Each time native code is removed, its block is added to the free list. A process called the *sweeper* is responsible for recycling blocks.

When a new native method is to be stored, the free list is searched for a block large enough to store the compiled code. If none is found, then, subject to the code cache having sufficient free space, a new block will be created from the unallocated space.

Native code can be removed from the code cache when:

- It is de-optimized (an assumption underpinning a speculative optimization turned out to be false).
- It is replaced with another compiled version (in the case of tiered compilation).
- The class containing the method is unloaded.

You can control the maximum size of the code cache using the VM switch:

```
-XX:ReservedCodeCacheSize=<n>
```

Note that with tiered compilation enabled, more methods will reach the lower compilation thresholds of the C1 client compiler. To account for this, the default maximum size is larger to hold these additional compiled methods.



In Java 8 on Linux x86-64 the default maximum sizes for the code cache are:

```
251658240 (240MB) when tiered compilation is enabled (-XX:+TieredCompilation)
50331648 (48MB) when tiered compilation is disabled (-XX:-TieredCompilation)
```

A single code cache can become fragmented—for example if many of the intermediate compilations from the C1 compiler are removed after they are replaced by C2 compilations. This can lead to the unallocated region being used up and all the free space residing in the free list.

The code cache allocator will have to traverse the linked list until it finds a block big enough to hold the native code of a new compilation. In turn, the sweeper will also have to do more work to scan for blocks that can be recycled to the free list.

In the end, any garbage collection scheme that does not relocate memory blocks will be subject to fragmentation, and the code cache is not an exception.

Without a compaction scheme, the code cache can fragment, and this can cause compilation to stop—it is just another form of cache exhaustion, after all. In Java 9, JEP 197 introduced the Segmented Code Cache, which aims to address the challenges of a single code cache. Tiered compilation has increased the amount of compiled code by approximately 200-400%. The code cache segmentation groups by compiled code types, closely tied to lifetime, helping to prevent the fragmentation effects and associated sweeps and collections.

This grouping has a locality benefit, improving access times. The groups of the Segmented Code Cache are:

- *Non-method code heap*, which contains non method compilations that will last in the code cache for the duration of the application. The size can be configured using `-XX:NonMethodCodeHeapSize`.
- *Profiled code heap*, which contains lightly optimized compiled code, which tends to have a short lifetime. The size can be configured using `-XX:ProfiledCodeHeapSize`.
- *Non-profiled code heap*, which contains fully optimized, non-profiled methods, which tend to have a long lifetime. The size can be configured using `-XX:NonProfiledCodeHeapSize`.

---

**WARNING**

These sizes are fixed, and adjusting them can have unexpected consequences if not fully tested with your application.

---

## Logging JIT Compilation

One important JVM switch that all performance engineers should know about is:

```
-XX:+PrintCompilation
```

This will cause a log of compilation events to be produced on `STDOUT` and will allow the engineer to get a basic understanding of what is being compiled.

For example, if the caching example from [Example 7-1](#) is invoked like this:

```
java -XX:+PrintCompilation optjava.Caching 2>/dev/null
```

Then the resulting log (under Java 21) will look something like this:

```
50  1  3  java.lang.Object::<init> (1 bytes)
55  2  3  java.lang.String::hashCode (60 bytes)
55  3  3  jdk.internal.util.ArraysSupport::signedHashCode (37 bytes)
56  5  3  java.util.ImmutableCollections$SetN::probe (56 bytes)
56  6  3  java.lang.Math::floorMod (20 bytes)
56  4  3  jdk.internal.util.ArraysSupport::vectorizedHashCode (158 bytes)
58  7  3  java.lang.StringLatin1::hashCode (52 bytes)
58  8  3  java.lang.String::equals (56 bytes)
58  9  3  java.lang.StringLatin1::equals (36 bytes)
58 11  4  java.lang.Object::<init> (1 bytes)
59 10  3  java.util.Objects::equals (23 bytes)
59 12  3  java.lang.module.ModuleDescriptor$Exports::<init> (20 bytes)
59  1  3  java.lang.Object::<init> (1 bytes)  made not entrant
59 13  3  java.util.Objects::requireNonNull (14 bytes)
59 16  3  java.util.Set::of (4 bytes)
59 14  3  java.util.AbstractCollection::<init> (5 bytes)
60 15  3  java.util.ImmutableCollections$AbstractImmutableCollection::<init> (5 bytes)
60 17  3  java.lang.module.ModuleDescriptor::modsHashCode (43 bytes)
60 18  3  java.util.Set::of (68 bytes)
61 19  3  java.lang.String::coder (15 bytes)
61 21  3  java.lang.String::length (11 bytes)
61 20  1  java.lang.module.ModuleDescriptor::name (5 bytes)
```

```

62  22  3  java.lang.String::isLatin1 (19 bytes)
62  23  1  java.lang.module.ModuleReference::descriptor (5 bytes)
65  24  3  java.lang.String::charAt (25 bytes)
66  25  3  java.lang.StringLatin1::charAt (15 bytes)
...

```

Note that as the vast majority of the JRE standard libraries are written in Java, they will be eligible for JIT compilation alongside application code. We should, therefore, not be surprised to see so many nonapplication methods present in the compiled code.

---

#### TIP

The exact set of methods that are compiled may vary slightly from run to run, even on a very simple benchmark. This is a side effect of the dynamic nature of PGO and should not be of concern.

---

The `PrintCompilation` output is formatted in a relatively simple way. First comes the time at which a method was compiled (in milliseconds since VM start). Next is a number that indicates the order in which the method was compiled in this run. Some of the other fields are:

*n*

Method is native

*s*

Method is synchronized

!

Method has exception handlers

⌘

Method was compiled via on-stack replacement

The level of detail available from `PrintCompilation` is somewhat limited. To access more detailed information about the decisions made by the HotSpot JIT compilers, we can use:

```
-XX:+LogCompilation
```

This is a diagnostic option we must unlock using an additional flag:

`-XX:+UnlockDiagnosticVMOptions`

This instructs the VM to output a logfile containing XML tags representing information about the queuing and optimization of bytecode into native code. The `LogCompilation` flag can be verbose and generate hundreds of megabytes of XML output.

However, as we will see in the next chapter, the open source `JITWatch` tool can parse this file and present the information in a more easily digestible format.

Other VMs, such as IBM's J9 with the Testarossa JIT, can also be made to log JIT compiler information, but there is no standard format for JIT logging, so developers must learn to interpret each log format or use appropriate tooling.

## Simple JIT Tuning

When undertaking a code tuning exercise, it is relatively easy to ensure that the application is taking advantage of JIT compilation.

The general principle of simple JIT compilation tuning: “Any method that wants to compile should be given the resources to do so.” To achieve this aim, follow this checklist:

1. First run the app with the `PrintCompilation` switch on.
2. Collect the logs that indicate which methods are compiled.
3. Now increase the size of the code cache via  
`ReservedCodeCacheSize`.
4. Rerun the application.
5. Look at the set of compiled methods with the enlarged cache.

The performance engineer will need to take into account the nondeterminism inherent in the JIT compilation. Keeping this in mind, a couple of obvious tells can easily be observed:

- Is the set of compiled methods larger in a meaningful way when the cache size is increased?
- Are all methods that are important to the primary transaction paths being compiled?

If the number of compiled methods does not increase (indicating that the code cache is not being fully utilized) as the cache size is increased, then provided the load pattern is representative, the JIT compiler is not short of resources.

At this point, it should be straightforward to confirm that all the methods that are part of the transaction hot paths appear in the compilation logs. If not, then the next step is to determine the root cause—why these methods are not compiling.

Effectively, this strategy is making sure that JIT compilation never shuts off by ensuring the JVM never runs out of code cache space. We will meet more sophisticated techniques later in the book, but this simple JIT tuning approach can help provide performance boosts for a surprising number of applications.

## Evolving Java Program Execution

At the start of this chapter, we introduced the typical lifecycle stages of a Java application. A graphical depiction of this process can be seen in [Figure 6-7](#).

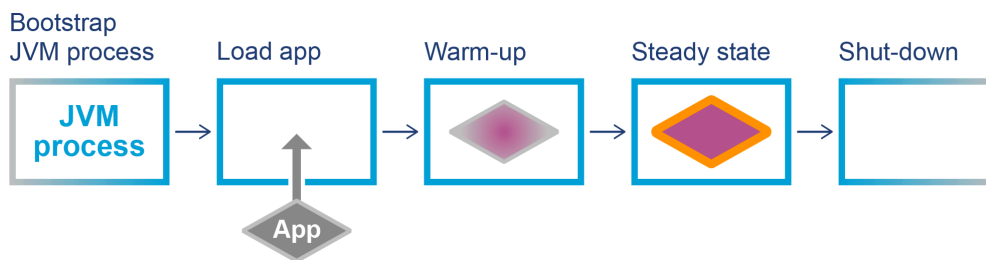


Figure 6-7. Simple lifecycle of a Java application

This represents the standard mental model for the overall lifecycle of Java applications and has been for as long as Java has had JIT compilation. However, it does have certain drawbacks—the major one being that execution time can be slower while the application transitions into steady state (i.e., “JVM warm-up”).

This transition time can easily run into the tens of seconds after application start. For long-running applications, this is not usually a problem—a process that is running continuously for hours (or days or weeks) receives far more benefit from the JIT compiled code than the one-off effort expended to create it at startup.

In the cloud native world, however, processes may be much shorter-lived. This raises several questions:

- Under what circumstances is the amortized cost of Java startup and JIT actually worth it?
- What could be done to make Java applications start up faster and avoid those costs?

- Are too many resources (especially memory) being used to provide dynamic capabilities that are not actually needed by many cloud native workloads?

To set the scene, let's first talk about AOT compilation and then move on to discuss one of the most popular modern frameworks that has been designed with these questions in mind—Quarkus.

## Ahead-of-Time (AOT) Compilation

If you have experience programming in languages such as C and C++, you will be familiar with AOT compilation (you may have just called it “compilation”). This is the process whereby an external program (the compiler) takes human-readable program source and outputs directly executable machine code.

---

### WARNING

Compiling your source code ahead of time means you have only a single opportunity to take advantage of any potential optimizations.

---

You will most likely want to produce an executable targeted to the platform and processor architecture you intend to run it on. These closely targeted binaries will be able to take advantage of any processor-specific features that can speed up your program.

However, in most cases, the executable is produced without knowledge of the specific platform that it will be executed upon. This means that AOT compilation must make conservative choices about which processor features are likely to be available. If the code is compiled with the assumption that certain features are available, and they turn out not to be, then the binary will not run at all.

This leads to a situation where AOT-compiled binaries usually do not take full advantage of the CPU's capabilities, and potential performance enhancements are left on the table.

The sophisticated JIT present in HotSpot does not suffer from these limitations. As we discussed at the start of this chapter, when HotSpot starts up, it probes the CPU to see exactly what instructions are available. With this information, the JVM can decide to enable processor-specific optimizations (referred to as *compiler intrinsics*) to tailor JIT compilation to the actual runtime environment in use. This is one of the mechanisms that means that Java applications can often get a performance boost just by upgrading the JVM they run on—without even recompiling the code.

New compiler intrinsics, and other enhancements, are developed all the time, and it is very common for applications to see noticeable gains, such as when upgrading the major version of the JVM used in production.

In general, Java applications have traditionally not been AOT-compiled.<sup>5</sup> There are a number of reasons for this, but chief among them is that the JVM is actually a very dynamic execution environment—which stands in contrast to the view of many developers that Java is a “static” language.

One particular problem that native compilation of Java applications runs into is how to handle *reflection*. Reflection is a dynamic, runtime mechanism that is very widely used in Java—many common frameworks and developer tools (such as debuggers and code browsers) rely upon reflection to implement their capabilities.

Fundamentally, Java applications can use reflection to load classes and to invoke methods whose names are unknown at compile time. This is an extremely powerful capability, and it can be used to implement very open and dynamic systems, but it can (and does) come into conflict with the more encapsulated systems that are desirable for AOT compilation and other cloud native techniques.

---

**NOTE**

A full discussion of reflection and related dynamic techniques can be found in the second edition of *The Well-Grounded Java Developer* by Benjamin J. Evans et al. (Manning, 2022).

---

Any AOT scheme for Java must eventually confront the fact that reflection and other dynamic techniques are everywhere in the ecosystem. We will have more to say about this subject when we discuss GraalVM later in this chapter.

Finally, we should note that AOT compilation is a *mechanism*, but what application owners and SREs actually care about is the performance of their applications—i.e., *outcomes*. It is a common error of technologists to confuse the two, especially when the former represents a fascinating intellectual challenge.

Let’s move on to meet Quarkus, an increasingly popular modern application development framework that focuses on cloud native deployment and can leverage, but does not require, AOT compilation.

# Quarkus

Quarkus is a well-established Java framework that has been developed by Red Hat and the community.<sup>6</sup> It is designed to be used in a cloud native environment, including microservices and serverless apps, and as such, it is optimized for fast startup and developer productivity.

Quarkus describes itself as a “Kubernetes Native Java stack tailored for OpenJDK HotSpot and GraalVM, crafted from the best of breed Java libraries and standards.” It implements standards such as the Jakarta EE and MicroProfile APIs as well as supporting emerging standards like OpenTelemetry.<sup>7</sup>

One of the mechanisms Quarkus uses for this is to introduce a new phase to the lifecycle of a Java application: the *build phase*. The aim of the build phase is to perform as much of the work that would normally be done at application startup as possible—effectively shifting computation from runtime to compilation time.

For example, dependency injection in Quarkus is based on ArC—a CDI-based dependency injection library that has been designed to fit the lifecycle of Quarkus applications.<sup>8</sup>

Quarkus also moves (“shifts left”) operations like classpath annotation scanning from runtime to build time. For this to work, we need to declare all dependencies at build time—Quarkus provides an indexing capability called Jandex to handle this, and a bytecode generation library (Gizmo) to reduce or remove the need for reflection.

This shift of computation makes sense in the cloud native world that we live in because of how applications are deployed these days, i.e., containers. In a container, new dependencies do not show up at runtime because container deployments are immutable. Furthermore, the shifting of code is actually beneficial to the C2 JIT because it leads to generation of Java code that is less dynamic, which makes C2’s job easier.

In production, Quarkus is capable of running in two modes: a dynamic VM mode that uses the HotSpot JVM in a traditional manner, and a *native mode* that uses an AOT capability provided by the GraalVM native image compiler (which we will discuss later in this chapter).

Developers sometimes assume that native mode is necessary to achieve fast startup of Quarkus applications. However, a surprising amount of startup computation can be shifted to build time—including such things as building the application’s object dependency graph. For example, starting a REST service in Java and responding to a request on a traditional



stack is approximately 4.3 seconds, Quarkus + JIT is 0.943 seconds, and native compilation is 0.016 seconds.<sup>9</sup>

The aggressive use of build-time shifting wherever possible means that Quarkus applications in dynamic VM mode typically start much faster than traditional Java systems. Native mode can provide some extra performance benefit, but exactly how much depends on the details of the application. Many teams find the startup and peak performance of Quarkus in dynamic VM mode to be excellent and do not feel the need to add the extra complexity required to build native-mode applications.

These two phases can be seen in [Figure 6-8](#).

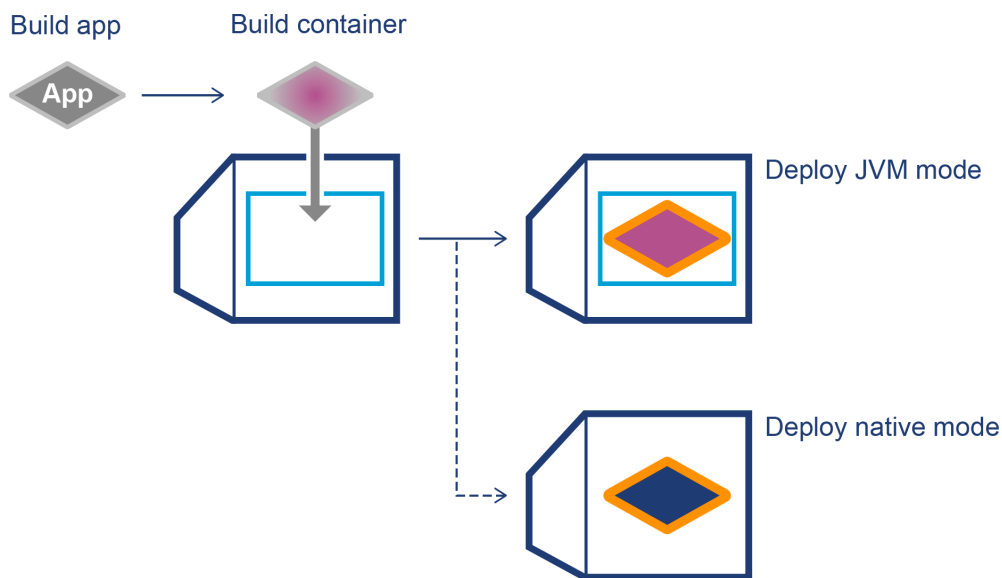


Figure 6-8. Quarkus build and deploy

Quarkus emphasizes developer productivity, and many teams have adopted Quarkus based primarily on the developer experience it provides.

For example, Quarkus provides a development mode, which can be as simple as running `./mvnw quarkus:dev` for a local application. This starts Quarkus with *live reload* and background compilation. This means that when you modify your Java files and/or resource files (including the configuration property file) and refresh your browser, your changes will automatically take effect.

The Quarkus framework enables both imperative and reactive styles of application development, which means that it can be easily adopted by teams migrating over from purely imperative models, such as Spring Boot. When an application requires high scalability, Quarkus also provides the capability to use reactive (i.e., nonblocking) programming.

With the flexibility provided by the choice of imperative and reactive programming styles, as well as both dynamic VM and native modes, it is per-

haps not surprising that Quarkus is being adopted by an increasing number of organizations and teams.

## GraalVM

[GraalVM](#) describes itself as “a high-performance JDK designed to accelerate the execution of applications written in Java and other JVM languages.”

Oracle developed GraalVM in its research division before shipping it as a product and currently provides two distinct distributions of the technology: Oracle GraalVM Community Edition (CE) and Oracle GraalVM Enterprise Edition (EE). The community edition is open source, but the enterprise edition is proprietary software that requires a paid license from Oracle to deploy in production.

GraalVM includes Truffle, a language implementation framework written in Java, to build interpreters for multiple programming languages, which then run on GraalVM.

Truffle is a fascinating technology, but perhaps more interesting for our purposes is the native image functionality. This is a key feature of GraalVM, implemented via the Graal compiler—a JIT compiler that is written in Java and can also operate as an AOT compiler.<sup>[10](#)</sup>

GraalVM’s native image has been steadily maturing for a number of years, and it is now deployed by an increasing number of teams who want to deploy Java applications but want to minimize startup time as much as possible.

One important caveat is that native image needs to know at build time the reflectively accessed program elements. It attempts to determine this automatically, by performing a static analysis that detects calls to the Reflection API. If the analysis fails, or the reflective code path is too complex, then the application’s build script must manually specify the elements that will be reflectively accessed at run time.

Quarkus can [leverage GraalVM’s native image capabilities](#) to produce native executables. Many developers and teams find that using Quarkus in native mode is easier than standalone GraalVM.

This is for several reasons, but the primary one is that compiling to native with Quarkus is much easier than building directly from scratch using GraalVM, because Quarkus has already done the heavy lifting of making the libraries work in native mode. Without framework help, getting production-quality builds from GraalVM can be challenging.

Red Hat recommends using the downstream Mandrel distribution of GraalVM CE, which is specifically tailored to working with Quarkus applications.

## Summary

The JVM's initial code execution environment is the bytecode interpreter. We have explored the basics of the interpreter, as a working knowledge of bytecode is essential for a proper understanding of JVM code execution. The basic theory of JIT compilation has also been introduced.

However, for most performance work, the behavior of JIT-compiled code is far more important than any aspect of the interpreter. For many applications, the simple tuning of the code cache shown in this chapter will be sufficient. Applications that are particularly performance-sensitive may require a deeper exploration of JIT behavior.

We have also discussed some initial aspects of specifically cloud native behavior, as it relates to the lifecycle of cloud-deployed Java applications.

In the next chapter, we will begin our journey into the deployment aspects that modern, cloud native Java applications require. The first building block of this will be a review of the relevant aspects of hardware and operating systems, which are still the bottom layer of application stacks.

- 1** Recent versions of the JVM also permit more exotic constants to support modern advanced VM techniques.
- 2** At least in mainstream server JVMs.
- 3** Modern versions of HotSpot may not need to check in between every bytecode, due to advances such as thread-local handshakes, but it remains a useful mental model for developers new to JVM internals.
- 4** This code is based on version 0.1.1 of [ocelotvm](#)—a partial implementation of a JVM interpreter designed for teaching.
- 5** There are exceptions to this, including the iOS App Store, which only permitted AOT-compiled Java apps, as the App Store rules forbid techniques such as JIT compilation.
- 6** Quarkus also supports Kotlin in core (and Scala via a community-supported Quarkiverse extension).
- 7** Jakarta EE is the successor to the Java EE standard, which was transferred to the Eclipse Foundation in 2017.

**8** ArC implements the CDI Lite specification.

**9** Data from <https://quarkus.io>.

**10** The Graal compiler works by essentially “compiling itself” to produce a native library that can be shipped as a JIT compiler for GraalVM.